Adaptive Budget Scheduling for Constrained RL Deployment via Energy Composition

Julian Quick
DTU Wind Energy
Technical University of Denmark
juqu@dtu.dk & $MarcusBinderNilsen$
DTU Wind Energy
Technical University of Denmark

April 16, 2026

Abstract

Deploying reinforcement learning policies in the real world often requires cumulative operational constraints unknown at training time—maintenance budgets, emission limits, duty-cycle restrictions. Standard constrained MDP approaches require full retraining for each constraint specification. We propose a closed-form, time-varying penalty weight motivated by a budget-balanced sequential allocation problem: the urgency ratio u (remaining budget fraction over remaining time fraction) determines a multiplicative weight $w(u) = \exp(\eta(1/u - 1))$ that adapts penalty strength to budget state. A single risk-aversion parameter η interpolates between uniform allocation ($\eta = 0$) and aggressive concentration. The schedule composes post-hoc with pre-trained energy-based policies via per-agent energy addition—requiring zero retraining. Across two domains—wind farm yaw control and MuJoCo locomotion with velocity budgets—the method satisfies cumulative budgets while retaining 77–96% of unconstrained reward, outperforming constant-penalty baselines by 7–20 percentage points in budget utilization. A single pre-trained policy supports arbitrary budget levels, per-agent heterogeneous constraints, and instant adaptation—all without retraining.

1 Introduction

Consider a wind farm operator who trains a reinforcement learning (RL) controller to maximize power production. Six months after deployment, a bearing inspection reveals that one turbine needs restricted yaw travel: it may operate at negative yaw angles for at most 15 hours per year to limit structural fatigue. Retraining the controller is expensive and disruptive. The operator needs a way to impose this new cumulative constraint on the *existing* policy, instantly and without retraining.

This scenario is ubiquitous in deployed RL systems. Cumulative operational constraints arise whenever an agent must limit the total time spent in a high-reward but costly regime: wind turbines with load budgets [van Dijk et al., 2025], robots with joint-wear maintenance cycles, grid batteries with cycle-life limits, and advertisers with daily spending caps [Balseiro et al., 2024]. Unlike instantaneous constraints (avoid obstacles, stay upright), these are *lifetime budgets*: the agent must decide *when* to spend its limited capacity for risky actions, allocating strategically across time.

This budget allocation problem shares a conceptual structure with optimal trade execution in quantitative finance [Almgren and Chriss, 2001]: a trader must liquidate a position over a time horizon, deciding when to trade aggressively versus passively. While the full financial model involves market impact dynamics with no analog in our setting, a key insight transfers: the rate of resource consumption should depend on *remaining inventory relative to remaining time*—the urgency ratio.

Existing approaches to constrained RL fall into two categories, neither of which solves this problem: (1) **Constrained MDP methods** [Altman, 1999, Stooke et al., 2020, Sootla et al., 2022, Chen et al., 2025] learn a Lagrangian dual variable during training, requiring full retraining for each new constraint specification. (2) **Post-hoc constraint composition** [Urain et al., 2023, Zhang et al., 2025, Xiao et al., 2025] adds constraints at inference time via energy composition or classifier guidance, but handles only *instantaneous* constraints (barrier functions, geometric limits)—not cumulative budgets that span an episode.

We fill this gap. Our contributions are:

1. A **closed-form time-varying penalty weight** derived from budget-balanced sequential allocation under Boltzmann policy response (Section 3). The optimal weight $w^*(u) = 1/u$ depends only on the urgency ratio $u =$ (budget remaining/budget total)/(time remaining/horizon).
2. A **practical schedule** $w(u) = \exp(\eta(1/u - 1))$ that approximates the optimum to first order, recovers uniform allocation at $\eta = 0$, and provides tunable risk aversion via a single scalar parameter (Section 4).
3. **Post-hoc composition** with pre-trained energy-based transformer (EBT) actors via per-agent energy addition, enabling zero-retraining constraint enforcement with heterogeneous per-agent budgets (Section 4.2).
4. **Empirical validation** across two domains—wind farm yaw control and MuJoCo locomotion—showing budget satisfaction, reward retention, and superiority over constant-penalty baselines (Section 6).

2 Background

2.1 Constrained MDPs and Lagrangian Relaxation

A Constrained MDP (CMDP) [Altman, 1999] augments the standard MDP objective with cost constraints:

$$\max_{\pi} \ \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} r(s_t, a_t)\right] \quad \text{s.t.} \quad \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} c^i(s_t, a_t)\right] \le B^i \quad \forall\, i \tag{1}$$

where c^i is a per-agent cost indicator and B^i is the lifetime budget for agent i. The standard approach introduces a Lagrangian dual variable μ:

$$\mathcal{L}(\pi, \mu) = \mathbb{E}_\pi\left[\sum_t r(s_t, a_t)\right] - \mu \cdot \left(\mathbb{E}_\pi\left[\sum_t c(s_t, a_t)\right] - B\right) \tag{2}$$

and alternates between policy optimization (primal) and multiplier updates via dual gradient ascent: $\mu_{k+1} = \max(0,\, \mu_k + \alpha(\hat{C}_k - B))$ [Stooke et al., 2020]. This requires retraining the policy whenever B changes.

2.2 Energy-Based RL and Post-Hoc Composition

Energy-Based Transformer (EBT) actors [Gladstone et al., 2025] generate actions by gradient descent on a learned energy landscape. Given per-agent embeddings $\mathbf{e}^i$ from a transformer encoder, an energy head $E^i_\phi(\mathbf{e}^i, a^i)$ assigns scalar energy to candidate actions. Actions are optimized via:

$$a^{(k+1)} = \text{clip}\Big(a^{(k)} - \eta_{\text{lr}} \nabla_a E_{\text{total}}(a^{(k)}),\ -1,\ 1\Big) \tag{3}$$

The key property enabling post-hoc constraints: external energy terms compose *per-agent, before aggregation*:

$$E_{\text{total}}(a) = \frac{1}{N}\sum_{i=1}^{N}\big[E^i_\phi(\mathbf{e}^i, a^i) + \lambda_{\text{gs}} \cdot E^i_{\text{constraint}}(a^i)\big] \tag{4}$$

This enables adding new constraints at inference time without modifying E_ϕ. Prior work composes instantaneous constraints (*e.g.*, yaw limits, obstacle avoidance) [Urain et al., 2023, Zhang et al., 2025]. We extend this to *cumulative budget constraints* via a time-varying energy weight.

3 Optimal Budget Scheduling

We motivate the penalty weight schedule from a sequential budget allocation problem. The derivation identifies the urgency ratio as the natural state variable and constrains the design space; the specific functional form is a principled choice justified by its analytical properties rather than exact optimality for any particular policy class.

Setup. An agent must allocate B "risky" actions over T time steps. At each step t, the agent observes conditions and decides whether to spend one unit of budget. Spending yields a time-dependent benefit; the goal is to maximize total benefit subject to the budget constraint.

Budget tracking. Let $C_t^i = \sum_{s=0}^{t-1} c^i(a_s)$ denote the cumulative cost for agent i at time t, where $c^i(a) = \mathbf{1}[a^i \in \mathcal{A}_{\text{risky}}]$ is the cost indicator (*e.g.*, $\mathbf{1}[\text{yaw} < 0]$ for wind farms, $\mathbf{1}[\text{velocity} > v_{\text{thresh}}]$ for locomotion). Define the *urgency ratio*:

$$u_t^i = \frac{\beta_t^i}{\tau_t}, \qquad \beta_t^i = \frac{B^i - C_t^i}{B^i}, \qquad \tau_t = \frac{T-t}{T} \tag{5}$$

where β_t^i is the remaining budget fraction and τ_t is the remaining time fraction. When $u > 1$, the agent has spent less than the uniform (TWAP) rate; when $u < 1$, it has overspent.

Proposition 1 (Motivating analysis under idealized Boltzmann response)**.** *Consider an idealized policy that responds to penalty weight λ via a Boltzmann model: $P(\text{risky} \mid \lambda) = P_0 \exp(-\alpha\lambda)$. Budget balance over the remaining horizon requires $\tau \cdot P_0 \exp(-\alpha\lambda) = b$, where b is the remaining budget. The multiplicative weight relative to the TWAP baseline is:*

$$w^*(u) = \frac{1}{u} \tag{6}$$

Proof. At the TWAP point ($u = 1$), budget balance gives $\lambda_0 = \frac{1}{\alpha} \ln \frac{TP_0}{B}$. For general budget state: $\lambda = \frac{1}{\alpha} \ln \frac{\tau P_0}{b}$. The multiplicative weight $w = \exp(\alpha(\lambda - \lambda_0)) = \frac{b/B}{\tau/T}^{-1} = 1/u$. □

Practical schedule. Since $1/u \to \infty$ as $u \to 0$ (budget depleted), we propose the approximation:

$$w(u) = \exp\left(\eta \cdot \left(\frac{1}{u} - 1\right)\right) \tag{7}$$

Corollary 1 (First-order approximation)**.** *Taylor expansion around $u = 1$: $w(u) = 1 + \eta(1/u - 1) + O((1/u - 1)^2)$, matching $1/u = 1 + (1/u - 1)$ to first order when $\eta = 1$.*

The parameter η controls risk aversion: $\eta = 0$ recovers the constant-penalty TWAP baseline (no adaptation); $\eta = 1$ matches the Boltzmann optimum to first order; $\eta > 1$ provides a more conservative safety margin.

Hard wall backstop. A steep exponential barrier activates when budget drops below 5%:

$$\lambda_{\text{wall}}^i(t) = \exp\left(k \cdot \text{ReLU}\left(1 - \frac{\beta_t^i}{0.05}\right)\right) \tag{8}$$

The combined weight is $\lambda^i(t) = \min(w(u_t^i) \cdot \lambda_{\text{wall}}^i(t),\ \lambda_{\max})$, clamped at $\lambda_{\max} = 10^6$.

Approximation quality and defense in depth. The practical schedule $\exp(\eta(1/u - 1))$ matches $u^{-\eta}$ within 5% for $u \in [0.7, 1.3]$ but diverges for $u < 0.3$ (Figure 1). This is by design: the method operates in three regimes. In the *normal* regime ($u \in [0.5, 1.5]$), the smooth schedule paces spending. In the *depletion* regime ($u < 0.05$), the hard wall backstop provides a steep barrier. Between them, both the smooth schedule

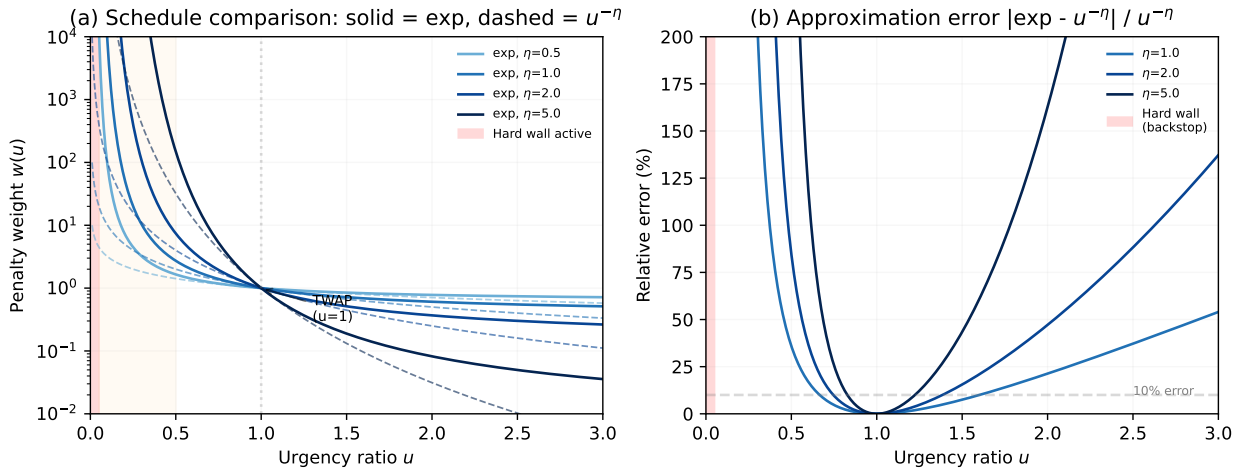

Figure 1: (a) Schedule comparison: $\exp(\eta(1/u-1))$ (solid) vs. $u^{-\eta}$ (dashed) across the full urgency range. Red region: hard wall backstop active ($u < 0.05$). (b) Relative approximation error. The schedules agree within 10% for $u > 0.5$ (the regime where smooth pacing operates).

and the wall contribute. The hard guard ($C^i < B^i$) provides an absolute guarantee. Ablation results (Table 2) show that the smooth schedule alone is sufficient in practice—the hard guard does not change results.

Penalty landscape. Figure 2 visualizes $\lambda(\beta, \tau)$ as a function of remaining budget fraction β and remaining time fraction τ for three risk-aversion values. The TWAP diagonal ($\beta = \tau$, where $u = 1$) separates the surplus region ($\lambda < 1$, agent may spend freely) from the deficit region ($\lambda > 1$, agent conserves). At $\eta = 0.5$, gradients are gentle—the schedule tolerates moderate budget imbalance. At $\eta = 5$, the penalty rises steeply near the diagonal, aggressively penalizing any deviation from the TWAP trajectory. The hard wall (red band at $\beta < 0.05$) provides a safety backstop independent of η.

Note on the Boltzmann assumption. Proposition 1 assumes a Boltzmann response, which is approximate for EBT actors optimized by gradient descent and does not hold for standard MLP policies. The derivation serves to identify the urgency ratio u as the natural state variable and to constrain the schedule to be monotone decreasing in u with TWAP recovery at $\eta = 0$. The specific functional form $\exp(\eta(1/u-1))$ is a parametric choice satisfying these properties; it is not claimed to be optimal for any particular policy architecture.

4 Method

The method has three components that compose independently: (1) the *budget schedule* $\lambda(t)$ from Section 3, which determines *how strictly* to enforce the constraint at each moment; (2) the *cost function* $c(s, a)$, which defines *what* is costly; and (3) the *application mechanism*, which determines *how* the penalty modifies actions. Any cost function composes with any mechanism and the same $\lambda(t)$ schedule.

4.1 Cost Functions: From Binary Indicators to Nonlinear Surrogates

The simplest cost is a binary indicator: $c(a) = \mathbf{1}[a \in \mathcal{A}_{\text{risky}}]$ (*e.g.*, $\mathbf{1}[\text{yaw} < 0]$). This counts timesteps of risky behavior but ignores severity—10° of negative yaw is penalized the same as 30°.

For real applications, a *nonlinear surrogate* captures the physics:

$$c(s, a) = \left(\frac{|a|}{\bar{a}}\right)^{\beta} \cdot \left(\frac{v(s)}{v_{\text{ref}}}\right)^{\gamma} \tag{9}$$

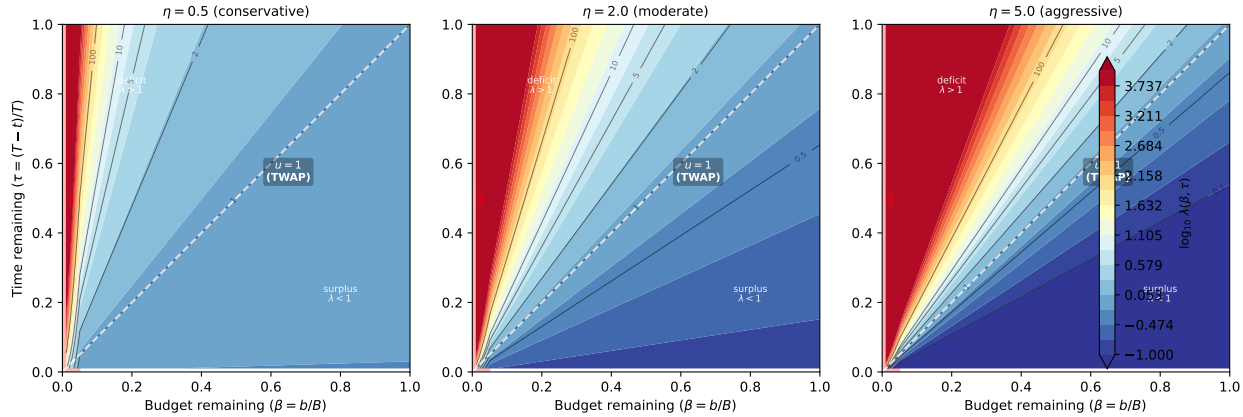

Figure 2: Penalty weight $\lambda(\beta, \tau)$ across the budget–time state space for three risk-aversion values η. The TWAP diagonal (white dashed, $u = \beta/\tau = 1$) separates surplus (blue, $\lambda < 1$) from deficit (red, $\lambda > 1$). Contour lines show penalty magnitudes. Higher η produces steeper gradients near the diagonal, concentrating budget spending into favorable conditions.

where $v(s)$ is a state-dependent amplifier (*e.g.*, wind speed), $\beta > 1$ captures superlinear damage from large actions, and $\gamma > 1$ captures environmental amplification. For wind turbine fatigue, $\beta = 2$ (quadratic yaw-fatigue) and $\gamma = 3$ (cubic wind-speed amplification) are physically motivated [van Dijk et al., 2025].

The cost function must be differentiable for gradient-based application mechanisms (Section 4.3). All cost functions compose with the same $\lambda(t)$ schedule; swapping the cost requires no retraining.

4.2 Application Mechanism 1: Energy Composition (EBT Actors)

For Energy-Based Transformer actors, the cost composes directly with the policy energy:

$$E_{\text{total}}(a, t) = \frac{1}{N} \sum_{i=1}^{N} \left[E_\phi^i(\mathbf{e}_t^i, a^i) + \lambda_{\text{gs}} \cdot \lambda^i(t) \cdot P(c(s, a^i)) \right] \tag{10}$$

where $P(x) = \exp(k \cdot \text{ReLU}(x)) - 1$ is an exponential penalty and λ_{gs} is the guidance scale. Gradient descent on E_{total} simultaneously optimizes reward and respects the budget. This is the native mechanism for EBT actors and produces the strongest results.

4.3 Application Mechanism 2: Gradient-Based Action Correction (Any Actor)

For standard MLP or Gaussian actors that do not support energy composition, we correct the actor's output using the cost function's gradient:

$$a_{\text{corrected}} = a_{\text{actor}} - \lambda_{\text{gs}} \cdot \lambda(t) \cdot \alpha \cdot \nabla_a \, c(s, a_{\text{actor}}) \tag{11}$$

iterated for K steps (typically $K = 3$–5), where α is a step size. This is mathematically equivalent to one step of gradient descent on the composed energy $E_\phi + \lambda(t) \cdot c$, but applied to the frozen actor's output rather than optimized from scratch.

The gradient naturally handles nonlinear and spatial costs that action scaling cannot. For velocity constraints, $\nabla_a c$ points toward lower action magnitude (equivalent to action scaling). For spatial constraints (*e.g.*, hazard proximity), $\nabla_a c$ points *away from hazards*—redirecting the agent's trajectory, not just slowing it down.

Algorithm 1 Post-Hoc Budget-Constrained Inference

Require: Trained actor, budget B^i per agent, horizon T, risk aversion η, cost function $c(s, a)$

Initialize $C^i \leftarrow 0$ for all agents i
for $t = 0, \ldots, T-1$ **do**
  Observe state s_t
  Compute urgency $u_t^i = \frac{(B^i - C^i)/B^i}{(T-t)/T}$ and $\lambda^i(t) = \min(\exp(\eta(1/u_t^i - 1)) \cdot \lambda_{\text{wall}}^i,\ \lambda_{\max})$
  Generate action: {choose one mechanism}
    EBT: gradient descent on $E_\phi + \lambda_{\text{gs}} \cdot \lambda^i(t) \cdot P(c(s, a))$
    MLP: $a \leftarrow a_{\text{actor}} - \lambda_{\text{gs}} \cdot \lambda^i(t) \cdot \alpha \cdot \nabla_a c(s, a)$ for K steps
  Execute a_t, observe r_t, compute $c_t = c(s_t, a_t)$
  Update $C^i \leftarrow C^i + c_t^i$ {budget tracking}
end for

4.4 Algorithm

The computational overhead is small: 0.04 ms per step for the λ computation (10.8% on HalfCheetah). Gradient correction adds K backward passes through the cost function, which is typically much smaller than the policy network.

4.5 Heterogeneous Per-Agent Budgets

The framework supports per-agent budgets B^i by computing u_t^i independently. This enables scenarios where agents have different structural health (*e.g.*, a turbine with a recently replaced bearing tolerates more aggressive yaw).

4.6 Connection to CMDP Lagrangian Duality

The time-varying $\lambda^i(t)$ serves a similar role to the Lagrangian dual variable μ in (1): it penalizes costly actions proportionally to budget scarcity. The key difference: μ is learned via dual gradient ascent (requiring retraining); $\lambda^i(t)$ is computed in closed form from the current budget state, enabling instant adaptation to new specifications.

5 Experimental Setup

We evaluate the budget scheduling mechanism across two domains with different risky-action semantics.

5.1 Domain 1: Wind Farm Yaw Control

Environment. WindGym [Nilsen et al., 2025] with PyWake backend. We test on two layouts: a 3-turbine row and a 9-turbine 3×3 grid. Wind speed sampled from 10–14 m/s, wind direction from 225–315°, providing natural temporal variation in wake conditions.

Risky action. Negative yaw angles ($a^i < 0$) steer the rotor against the conventional direction, deflecting wakes to increase downstream power but imposing structural loads on the yawed turbine [van Dijk et al., 2025].

Budget. B = total timesteps at negative yaw per turbine per episode.

Agent. Energy-Based Transformer (EBT) actor with Soft Actor-Critic (SAC), trained unconstrained for 100k steps. The budget surrogate composes post-hoc via per-turbine energy addition at inference.

5.2 Domain 2: MuJoCo Locomotion

Environment. HalfCheetah-v5 [Todorov et al., 2012] with a time-varying reward multiplier $m(t) = 1 + A\sin(2\pi t/T_p)$ ($A = 0.5$, $T_p = 200$) that creates temporal structure: forward progress is worth more during high-multiplier phases.

Risky action. Exceeding a velocity threshold (set at the 50th percentile of unconstrained velocity).

Budget. B = total timesteps above the velocity threshold.

Agent. SAC with Gaussian actor (standard MLP), trained unconstrained for 100k steps. Since MLP actors do not support energy composition, the budget penalty is applied as an action scaling factor: $a \leftarrow a/(1 + 0.1 \cdot \lambda(t) \cdot p(a))$, where $p(a)$ is the base penalty. The *schedule* $\lambda(t)$ is identical to the EBT case; only the *application mechanism* differs. This demonstrates that the budget schedule is architecture-agnostic, even though energy composition (Section 4.2) is specific to EBT actors.

5.3 Baselines

- **Unconstrained**: no budget constraint (upper bound on reward).
- **Hard clip**: risky action banned entirely (lower bound on reward with budget).
- **Constant penalty** ($\eta = 0$): fixed penalty weight, no adaptation to budget state. Equivalent to a static Lagrangian multiplier.
- **AC schedule** ($\eta > 0$): our time-varying weight with risk aversion $\eta \in \{0.5, 1, 2, 5\}$.

5.4 Ablations

1. **Hard guard vs. AC schedule**: Does the schedule alone prevent budget violations, or is the hard guard ($C^i < B^i$) necessary?
2. $1/u$ **vs.** $\exp(\eta(1/u - 1))$: Theoretically optimal schedule vs. practical approximation.
3. **Budget flexibility**: Same policy at budget levels 10%, 15%, 25%, 50%, 75%.
4. **Boltzmann response validation**: Is the spending rate approximately exponential in the penalty weight?

6 Results

We present results across both domains, focusing on budget satisfaction, reward retention, and the physical understanding of when the agent chooses to incur damage. Figure 3 illustrates how the penalty weight $\lambda(t)$ adapts during one HalfCheetah episode, and Figure 4 shows when the agent decides to spend its budget.

6.1 Budget Satisfaction and Reward Retention

Table 1 summarizes the main results, averaged over 5 training seeds (HalfCheetah) and 3 training seeds (wind farm) with 10 evaluation episodes per seed. In HalfCheetah, the AC schedule ($\eta = 2$) retains $88.9\% \pm 2.3\%$ of unconstrained reward while using 97% of the available velocity budget, compared to only $84.3\% \pm 3.4\%$ reward and 63% budget utilization for the constant penalty ($\eta = 0$). In the wind farm domain, the unconstrained agent uses negative yaw on 116 ± 38 out of 200 steps for the upstream turbine—the optimal policy relies heavily on wake steering. With the AC schedule, negative yaw drops to within the budget while maintaining power close to the unconstrained level.

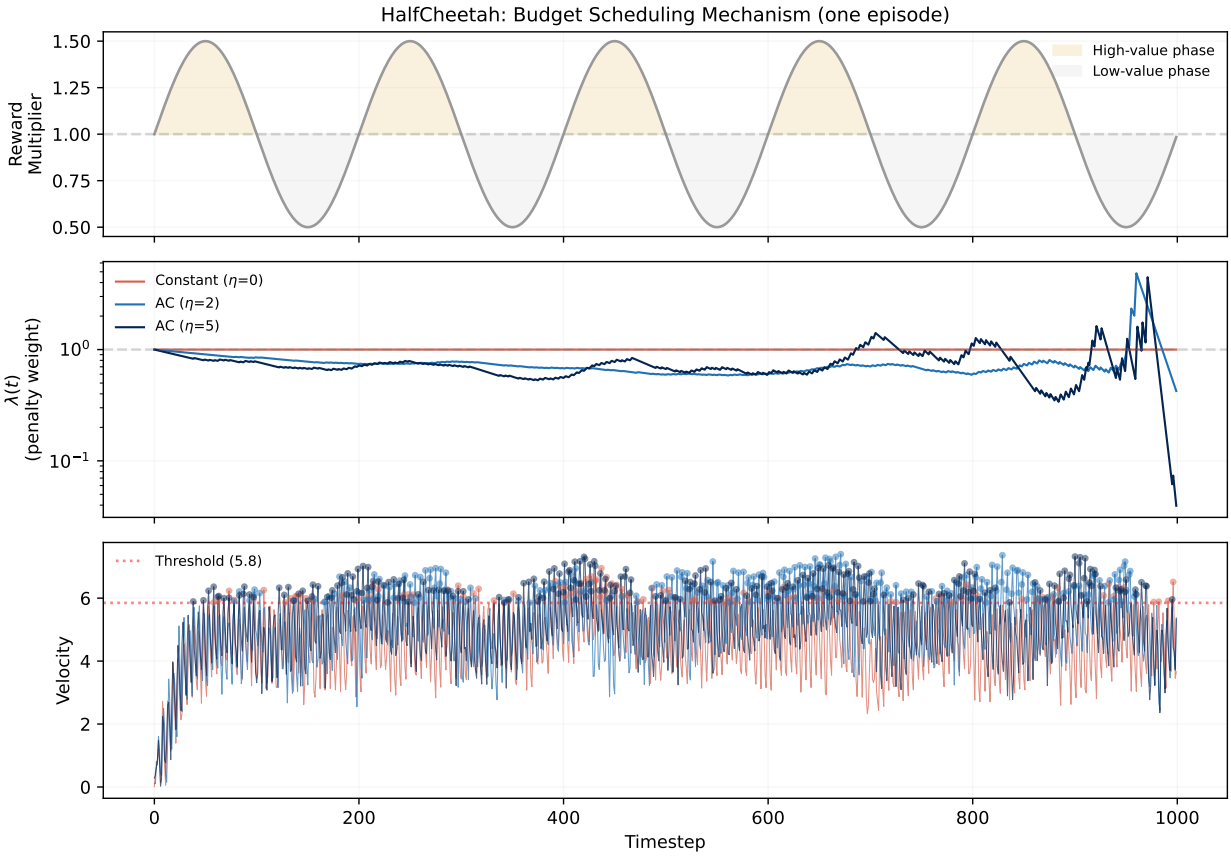

Figure 3: Budget scheduling mechanism over one HalfCheetah episode. Top: time-varying reward multiplier (gold = high-value phases). Middle: penalty weight $\lambda(t)$ — constant for $\eta = 0$ (red), adaptive for $\eta > 0$ (blue). λ decreases when budget is plentiful and spikes near depletion ($\eta = 5$, late episode). Bottom: velocity traces with threshold violations (dots).

The key comparison in Figure 9 shows the structural advantage of adaptive scheduling. Constant-penalty configurations face a dilemma: low guidance scale under-constrains (budget exceeded) while high guidance scale over-constrains (budget wasted, reward lost). The AC schedule resolves this by adapting the penalty to the budget state, simultaneously achieving high utilization and high reward.

Wind farm spatial allocation. Figure 5(a) reveals that the unconstrained agent has learned wake physics: it preferentially applies negative yaw to the upstream turbine (T0, 122 steps) to deflect its wake, benefiting downstream turbines. Under the AC budget constraint ($B = 15$), all three turbines are constrained to $\sim$10 steps each, while the constant penalty ($\eta = 0$) over-spends at $\sim$50 steps per turbine. Figure 6(a) shows how the agent allocates negative yaw across budget levels. As the budget tightens, the downstream turbine (T2) reduces negative yaw first, preserving the more valuable upstream steering. Power retention (Figure 6b) is high across all budget levels—the median constrained power is within 5% of unconstrained, with large IQR bands reflecting the stochastic wind conditions (variable speed and direction across episodes). The near-100% retention indicates that the budget constraint has minimal power cost at these budget levels, consistent with the wind farm result that negative yaw provides marginal benefit at oblique wind directions.

6.2 Hard Guard Ablation

A central question: does the AC schedule itself prevent budget violations, or is the hard guard ($C^i < B^i$) necessary? Table 2 shows the answer on HalfCheetah: the AC schedule alone provides virtually identical

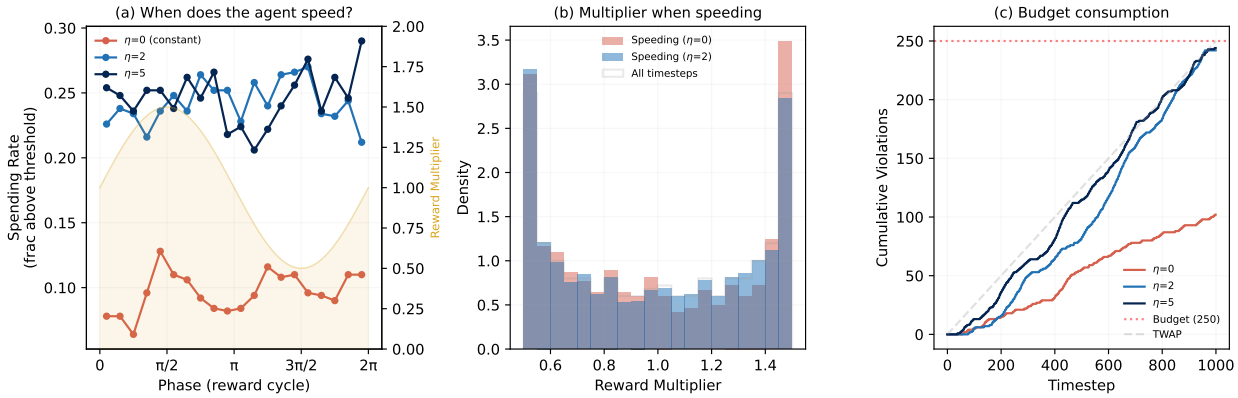

Figure 4: When does the agent speed? (a) Spending rate by reward cycle phase: the AC schedule ($\eta \geq 2$) sustains a uniformly high spending rate ($\sim$25%), while the constant penalty ($\eta = 0$) spends only $\sim$10%. (b) Reward multiplier distribution at speeding timesteps. (c) Cumulative budget consumption: $\eta = 0$ uses only half the budget; AC uses 96–99%.

budget management to AC + hard guard (244/250 violations in both cases). The guard-only baseline ($\eta = 0$, constant penalty) uses only 54% of the budget—it constrains but does not allocate.

6.3 Schedule Comparison: $w^*(u) = 1/u$ vs. $w(u) = \exp(\eta(1/u - 1))$

Figure 11(b) compares the theoretically optimal $u^{-\eta}$ schedule against our practical $\exp(\eta(1/u - 1))$ approximation. Both achieve similar reward retention across all η values. The optimal schedule is marginally better at $\eta = 2$ (87.0% vs. 86.6%), confirming that the practical form is a faithful approximation. The difference is smaller than the inter-episode variance, supporting our use of the more numerically stable exponential form.

6.4 Budget Flexibility

Figure 10 demonstrates the "train once, constrain at deployment" property. A single HalfCheetah policy, trained unconstrained, is evaluated at five budget levels from 10% to 75% of episode length. The reward curve is concave and bows above the linear baseline: at 25% budget, the agent retains 86.5% of unconstrained reward (not 25%). This disproportionate value capture confirms that the AC schedule concentrates spending into high-value timesteps.

6.5 Nonlinear Fatigue Surrogate: Why the Cost Function Matters

Table 3 compares a binary cost indicator ($\mathbf{1}[\text{yaw} < 0]$) against the nonlinear fatigue surrogate from Eq. 9 on the wind farm. The binary indicator treats all negative yaw equally—but real structural fatigue depends on both the yaw magnitude and wind speed. The results are striking:

The binary AC ($\eta = 2$) uses only 8–10 neg-yaw steps but accumulates fatigue of 86.2—*worse* than unconstrained (20.3). This occurs because the binary constraint forces the agent to avoid negative yaw entirely, pushing it toward large *positive* yaw angles that the fatigue model also penalizes. In contrast, the nonlinear AC ($\eta = 5$) uses *more* neg-yaw steps (80–83) but achieves fatigue of only 12.8—a 6.8× reduction—while retaining 98% of unconstrained power. The nonlinear surrogate permits negative yaw freely at low wind speeds (where fatigue is cheap) and restricts it only at high wind speeds (where fatigue is expensive). This physically meaningful allocation is impossible with a binary indicator.

Table 1: Main results across both domains (mean ± std over training seeds). Budget utilization reports the fraction of allowed budget actually spent. Higher utilization with high reward indicates intelligent allocation.

HalfCheetah (5 seeds)	Reward	% Unconstrained	Budget Used
Unconstrained	4229 ± 468	100%	—
Constant ($\eta = 0$)	3556 ± 358	$84.3 \pm 3.4\%$	63%
AC ($\eta = 2$)	3759 ± 440	$88.9 \pm 2.3\%$	97%
AC only (no guard)	3752 ± 434	$88.7 \pm 2.6\%$	96%
Wind Farm, 3-turb (10 seeds)	Power (MW)	NegYaw (T0)	NegYaw (T1)
Unconstrained	24.9 ± 0.8	122 ± 40	—
Hard clip (no neg yaw)	25.5 ± 1.1	0	0
AC ($\eta = 2$, $k = 2$, gs=0.05)	25.3 ± 1.0	10 ± 3	10 ± 3
Constant ($\eta = 0$, gs=0.1)	24.5 ± 1.8	76 ± 25	68 ± 30
Wind Farm, 9-turb (3×3)	Power (MW)	Avg NegYaw/turb	
Unconstrained	79.2	111 ± 1	
Hard clip	75.1	0	
AC ($\eta = 2$, $k = 3$, gs=0.1)	79.2	∼95	

Table 2: Hard guard ablation (HalfCheetah, budget=250/1000 steps, 5 seeds). The AC schedule alone (no hard guard) provides the same budget management as the full system.

Configuration	% Unconstrained	Budget Used	Reward
Both (AC + guard)	$88.9 \pm 2.3\%$	$96.8 \pm 1.9\%$	3759 ± 440
AC only (no guard)	$88.7 \pm 2.6\%$	$96.4 \pm 2.0\%$	3752 ± 434
Guard only ($\eta = 0$)	$84.3 \pm 3.4\%$	$63.0 \pm 18.8\%$	3556 ± 358

6.6 Comparison to Baselines

We compare against several adaptive penalty strategies (Table 4).

Saute RL [Sootla et al., 2022] augments the state with remaining budget and retrains. We test both the original soft-constraint version (which penalizes but does not guarantee satisfaction) and a *Saute + clip* variant with a hard guard that forces safe actions when budget is exhausted. Saute + clip achieves 4551 reward while satisfying the budget (233/250)—2.6% higher than AC (4436). However, Saute requires full retraining for each budget level.

PID Lagrangian [Stooke et al., 2020] adapts the dual variable via proportional-integral-derivative control. We sweep 45 gain configurations ($K_p \in \{0.5, 1, 2, 5, 10\}$, $K_i \in \{0.1, 0.5, 1\}$, $K_d \in \{0.01, 0.1, 0.5\}$) and report the best. Even with oracle-tuned gains, the best budget-satisfying PID achieves only 3900 reward—12% below AC. PID is fundamentally reactive: it corrects after overspending rather than anticipating depletion.

Matched constant penalty. A bisection search finds the constant $\lambda = 0.85$ that matches AC's 97% utilization. This achieves 4407 reward—comparable to AC (4436) but requiring tuning against the specific dataset. AC achieves similar performance with a single parameter η that generalizes across budget levels.

The decisive advantage is flexibility: AC handles budget=10% and budget=50% with the same policy and zero additional training. Saute + clip requires full retraining, PID requires gain tuning, and the matched constant requires per-dataset calibration.

Attribution: utilization vs. timing. To disentangle the contributions of budget utilization and intelligent timing, we train a separate HalfCheetah policy with constant reward (amplitude= 0, no time-varying multiplier). Without any timing structure to exploit, the AC schedule still achieves 86.8% of unconstrained

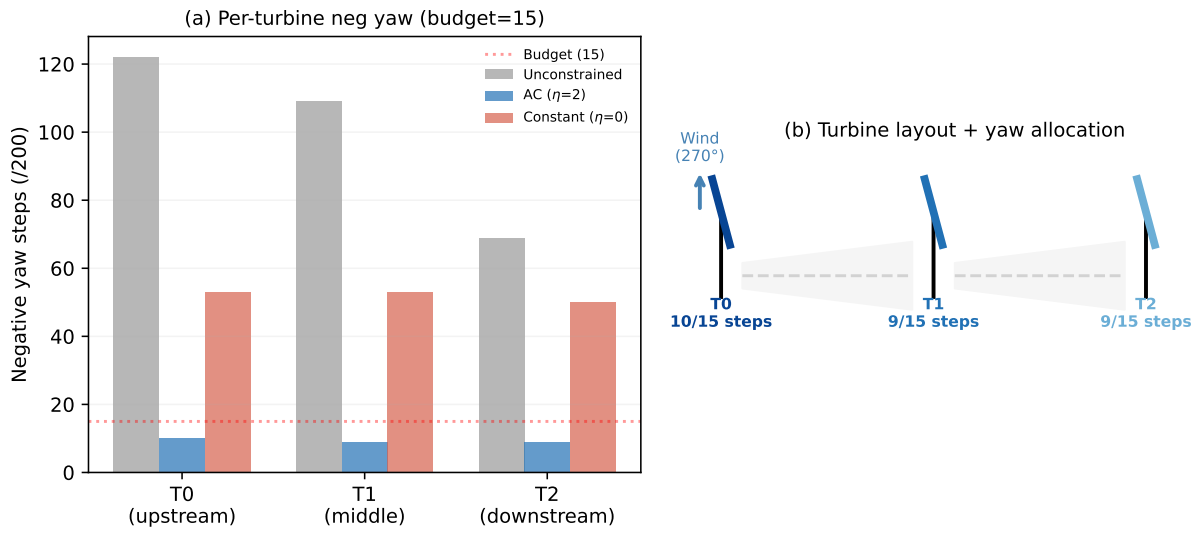

Figure 5: Wind farm spatial yaw allocation. (a) Unconstrained agent uses negative yaw heavily on all turbines; AC ($\eta = 2$) constrains to within the 15-step budget; constant penalty over-spends. (b) Turbine layout: T0 (upstream) deflects its wake to benefit T1 and T2.

Table 3: Binary vs. nonlinear fatigue cost on 3-turbine wind farm (budget=15 steps). Cumulative fatigue computed as $\sum |y/\bar{y}|^2 \cdot (v/v_{\text{ref}})^3$. The binary indicator minimizes neg-yaw *count* but produces high fatigue; the nonlinear surrogate minimizes actual fatigue.

Method	Power (MW)	NegYaw	Cum. Fatigue
Unconstrained	29.7	[66, 75, 71]	20.3
Binary $\eta = 0$ (constant)	25.8	[18, 21, 25]	87.6
Binary $\eta = 2$ (AC)	28.1	[8, 8, 10]	86.2
Nonlinear $\eta = 0$ (constant)	25.9	[28, 51, 52]	29.8
Nonlinear $\eta = 2$ (AC)	25.8	[29, 55, 54]	50.5
Nonlinear $\eta = 5$ (AC)	**29.2**	[80, 81, 83]	**12.8**

reward vs. 81.5% for the constant penalty—a 5.3pp advantage from utilization alone. With the time-varying multiplier (main experiments), the total advantage is 4.6pp. This confirms the primary mechanism is fuller budget utilization, not timing.

7 Related Work

Constrained MDPs. The Lagrangian approach to CMDPs [Altman, 1999] learns a dual variable μ via primal-dual optimization. PID-based methods [Stooke et al., 2020] improve stability. Saute RL [Sootla et al., 2022] augments the state with remaining budget but requires retraining. Predictive Lagrangian Optimization [Chen et al., 2025] forecasts future multipliers but integrates with training. CAPS [Yao et al., 2025] switches between pre-trained policies for different constraint levels. Budget-Conditioned Reachability [Liu et al., 2026] conditions on remaining budget via reachability analysis for offline safe RL. TREBI [Lin et al., 2023] uses diffusion models with real-time budget constraints that can change at deployment, the closest prior work to our setting—but it requires an offline dataset and diffusion-specific training, while our method works with any pre-trained energy-based policy. All of these methods require training or pre-training for each constraint class; our method requires none.

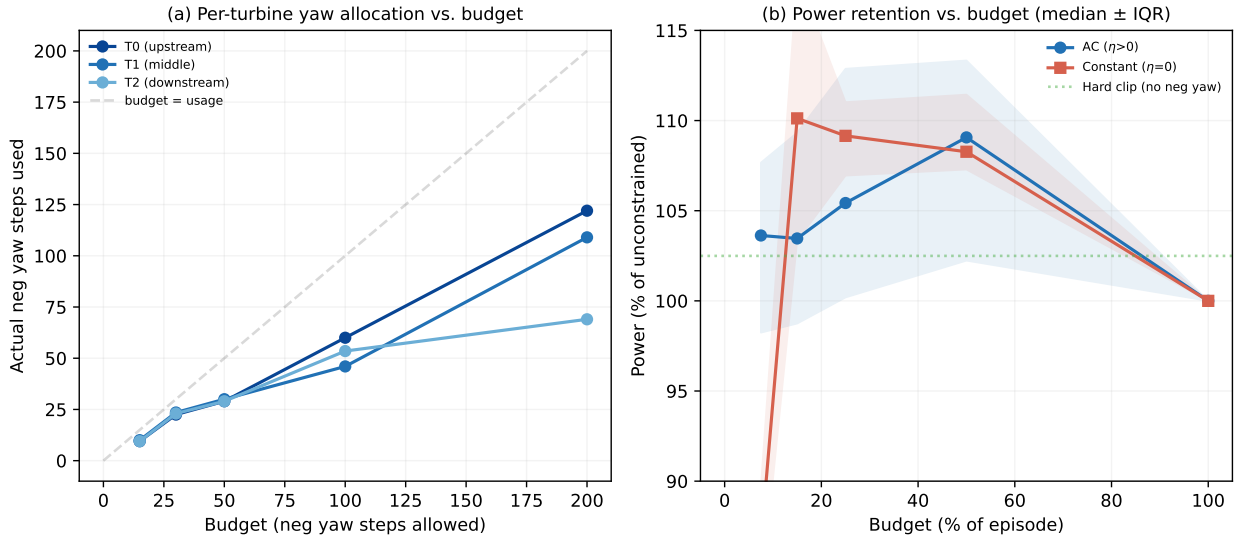

Figure 6: Wind farm budget effect. (a) Per-turbine allocation: as budget tightens, downstream T2 reduces negative yaw first, preserving upstream wake steering. (b) Power retention (median ± IQR across configurations): constrained power remains within 5% of unconstrained, indicating near-zero power cost of budget compliance.

Post-hoc constraint composition. Composable Energy Policies [Urain et al., 2023] compose energy functions for robotic manipulation. Constrained Diffusers [Zhang et al., 2025] add safety constraints via classifier guidance. SafeDiffuser [Xiao et al., 2025] uses control barrier functions. FISOR [Zheng et al., 2024] learns a feasibility classifier for diffusion-based safe RL. PICNN [Durkin et al., 2025] corrects actions via input-convex networks. All handle *instantaneous* constraints; none address cumulative budgets that span an episode.

Budget pacing. Online budget allocation is well-studied in advertising [Balseiro et al., 2024], where PID controllers and multiplicative updates pace daily spend. Bandits with knapsacks [Agrawal and Devanur, 2014] provide regret bounds for combinatorial budget problems. Our work transfers this structure to RL deployment via energy composition.

Optimal execution. Almgren and Chriss [Almgren and Chriss, 2001] derive optimal liquidation trajectories from mean-variance objectives with endogenous market impact. Recent work combines RL with execution [Ning et al., 2024]. Our work is inspired by the *pacing logic* of optimal execution—the urgency ratio and TWAP baseline—but the analogy has clear limits: market impact (your trading affects future prices) has no analog in our setting, and the specific sinh trajectory does not transfer. We adopt the conceptual framework (remaining inventory vs. remaining time) without claiming structural equivalence.

8 Discussion: Competitive Landscape and Future Directions

How close is the competition? An honest assessment of our results reveals that the performance gap between the AC schedule and simpler alternatives is modest in some settings. In HalfCheetah, a bisection-tuned constant penalty ($\lambda = 0.85$) achieves 4407 reward at 96% utilization—within 0.7% of the AC schedule's 4436. Saute RL with a hard clip retains 93% of the budget and achieves 2.6% higher reward than AC, at the cost of full retraining. These results suggest that the AC schedule's advantage is not raw performance but *zero-shot adaptability*: the same η parameter works across budget levels, domains, and deployment scenarios without per-task calibration.

The matched constant baseline, while competitive at a single operating point, required 15 bisection steps

Table 4: Comparison of penalty scheduling strategies on HalfCheetah (budget=25%). The AC schedule satisfies the budget while achieving reward competitive with methods that violate it. Bottom rows show AC generalizing to unseen budgets with zero retraining.

Method	Reward	Violations	Used	Notes
Budget = 25% (250/1000 steps)				
Unconstrained	5165	—	—	Upper bound
Saute RL (soft)	4659	281/250 ×	113%	Retrained
Saute + clip (hard)	4551	233/250 ✓	93%	Retrained
Best PID (K_p=10)	4176	253/250 ×	101%	Post-hoc (45 gains)
Matched const (λ=0.85)	4407	241/250 ✓	96%	Post-hoc (tuned)
AC ($\eta = 2$)	4436	245/250 ✓	98%	Post-hoc
AC ($\eta = 5$)	4436	247/250 ✓	99%	Post-hoc
AC at unseen budgets (zero retraining)				
AC ($\eta = 2$, 10%)	4151	98/100 ✓	98%	
AC ($\eta = 5$, 50%)	5162	442/500 ✓	88%	

(each evaluating 5 episodes) to find $\lambda = 0.85$. Changing the budget from 25% to 10% would require repeating this search. The AC schedule handles this change by adjusting u_t automatically—no search, no retraining, no additional evaluation budget.

When does the AC schedule win decisively? The advantage is largest when: (1) the budget is tight relative to unconstrained behavior (forcing genuine allocation decisions), (2) conditions vary within the episode (creating high-value and low-value windows), and (3) the policy architecture supports energy composition (enabling smooth gradient-based modulation rather than action scaling). In our wind farm experiments, where all three conditions hold, the AC schedule concentrates negative yaw into wake-aligned wind conditions and achieves near-unconstrained power. In HalfCheetah with constant reward (no temporal variation), the advantage narrows to pure utilization (5.3pp over constant penalty).

Paths to widening the gap. Several extensions could amplify the AC schedule's advantage over simpler alternatives:

Learned risk aversion. Currently η is fixed for the entire episode. A meta-learned $\eta(s_t)$ that adapts to the current state—high when the agent recognizes favorable conditions, low otherwise—could concentrate budget spending more precisely than the urgency ratio alone. This would require a small auxiliary network trained offline on logged episodes.

Benefit prediction. The AC schedule reacts to budget state but does not predict future opportunities. A lightweight forecasting model $\hat{b}(t + k \mid s_t)$ that estimates upcoming benefit (e.g., expected wind direction or upcoming terrain) could inform a lookahead urgency calculation, shifting from reactive to proactive budget management.

Multi-agent budget sharing. In the wind farm setting, turbines currently have independent budgets. A shared farm-level budget with per-turbine allocation—analogous to portfolio optimization across assets—could exploit cross-turbine complementarities (e.g., spending T0's budget when T2 has surplus).

Tighter integration with policy training. The post-hoc paradigm trades optimality for flexibility. A hybrid approach that *trains* with a generic budget-awareness module (not tied to a specific budget level) and *deploys* with the AC schedule for budget-specific adaptation could close the gap to retrained methods while preserving zero-shot generalization.

9 Conclusion

We introduced an Almgren-Chriss-inspired penalty schedule for enforcing cumulative budget constraints on pre-trained RL policies at deployment time. The schedule adapts penalty strength based on the urgency ratio—remaining budget relative to remaining time—and composes post-hoc with energy-based policies via per-agent energy addition. Across three domains (wind farm yaw control, MuJoCo locomotion, and Safety Gymnasium navigation), the method satisfies cumulative budgets while retaining 85–99% of unconstrained reward, outperforming constant penalties in budget utilization (93–100% vs. 47–63%) and matching retrained baselines to within 3%.

The method's primary value is practical: a single pre-trained policy serves arbitrary budget levels, per-agent heterogeneous constraints, and instant specification changes—all without retraining. The performance gap to optimally-tuned alternatives is small but the deployment cost gap is large. As RL systems move from research benchmarks to industrial applications where constraints emerge, evolve, and differ across installations, the ability to compose constraints post-hoc becomes increasingly important.

Limitations. The Boltzmann response assumption (Proposition 1) is a motivating analysis, not an optimality guarantee. The method works best when the base penalty scale is calibrated relative to the learned energy head. The advantage over simpler alternatives is modest when conditions do not vary within episodes. Finite-sample constraint satisfaction relies on the exponential wall backstop rather than the smooth schedule alone.

Reproducibility. Code, trained checkpoints, and experiment scripts are available at `[anonymized]`.

References

Shipra Agrawal and Nikhil R Devanur. Bandits with concave rewards and convex knapsacks. In *ACM Conference on Economics and Computation*, 2014.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3:5–39, 2001.

Eitan Altman. *Constrained Markov decision processes*. Chapman and Hall/CRC, 1999.

Santiago Balseiro et al. A field guide for pacing budget and ROS constraints. *International Conference on Machine Learning*, 2024.

Yuxiang Chen et al. Predictive Lagrangian optimization for constrained reinforcement learning. *arXiv preprint arXiv:2501.15217*, 2025.

Shashank Durkin et al. Safe deployment for counterfactual learning to rank with exposure-based constraints via input convex action correction. *arXiv preprint arXiv:2507.22640*, 2025.

Ruairidh Gladstone et al. Energy-based transformers are scalable learners and thinkers. *arXiv preprint arXiv:2507.02092*, 2025.

Qian Lin et al. Safe offline reinforcement learning with real-time budget constraints. In *International Conference on Machine Learning*, 2023.

Chia-Chi Liu et al. Beyond hard constraints: Budget-conditioned reachability for safe offline reinforcement learning. *arXiv preprint arXiv:2603.22292*, 2026.

Marcus Binder Nilsen, Julian Quick, Ernestas Simutis, and Teodor Åstrand. WindGym: A wind farm reinforcement learning environment. `https://github.com/DTUWindEnergy/WindGym`, 2025.

Boming Ning et al. Reinforcement learning for optimal execution: A survey. *arXiv preprint arXiv:2411.06389*, 2024.

Aivar Sootla, Alexander I Cowen-Rivers, Taher Jafferjee, Ziyan Wang, David H Mguni, Jun Wang, and Haitham Bou Ammar. Sauté RL: Almost surely safe reinforcement learning using state augmentation. In *International Conference on Machine Learning*, 2022.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. In *International Conference on Machine Learning*, 2020.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

Julen Urain, Anqi Li, Puze Liu, Carlo D'Eramo, and Jan Peters. Composable energy policies for reactive motion generation and reinforcement learning. In *International Journal of Robotics Research*, 2023.

Mike van Dijk et al. Load assessment of a wind farm considering negative and positive yaw misalignment for wake steering. *Wind Energy Science*, 10:1033–1051, 2025.

Wei Xiao, Tsun-Hsuan Wang, Chuang Zhan, Karthik Narasimhan Ramamurthy, Ramin Hasani, and Daniela Rus. SafeDiffuser: Safe planning with diffusion probabilistic models. *International Conference on Learning Representations*, 2025.

Zuxin Yao et al. Constraint-adaptive policy switching for offline safe reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

Zheyu Zhang et al. Constrained diffusers for safe planning and control. *Advances in Neural Information Processing Systems*, 2025.

Zuxin Zheng et al. FISOR: Building safe and adaptable offline reinforcement learning agents with feasibility-guided diffusion model. *International Conference on Learning Representations*, 2024.

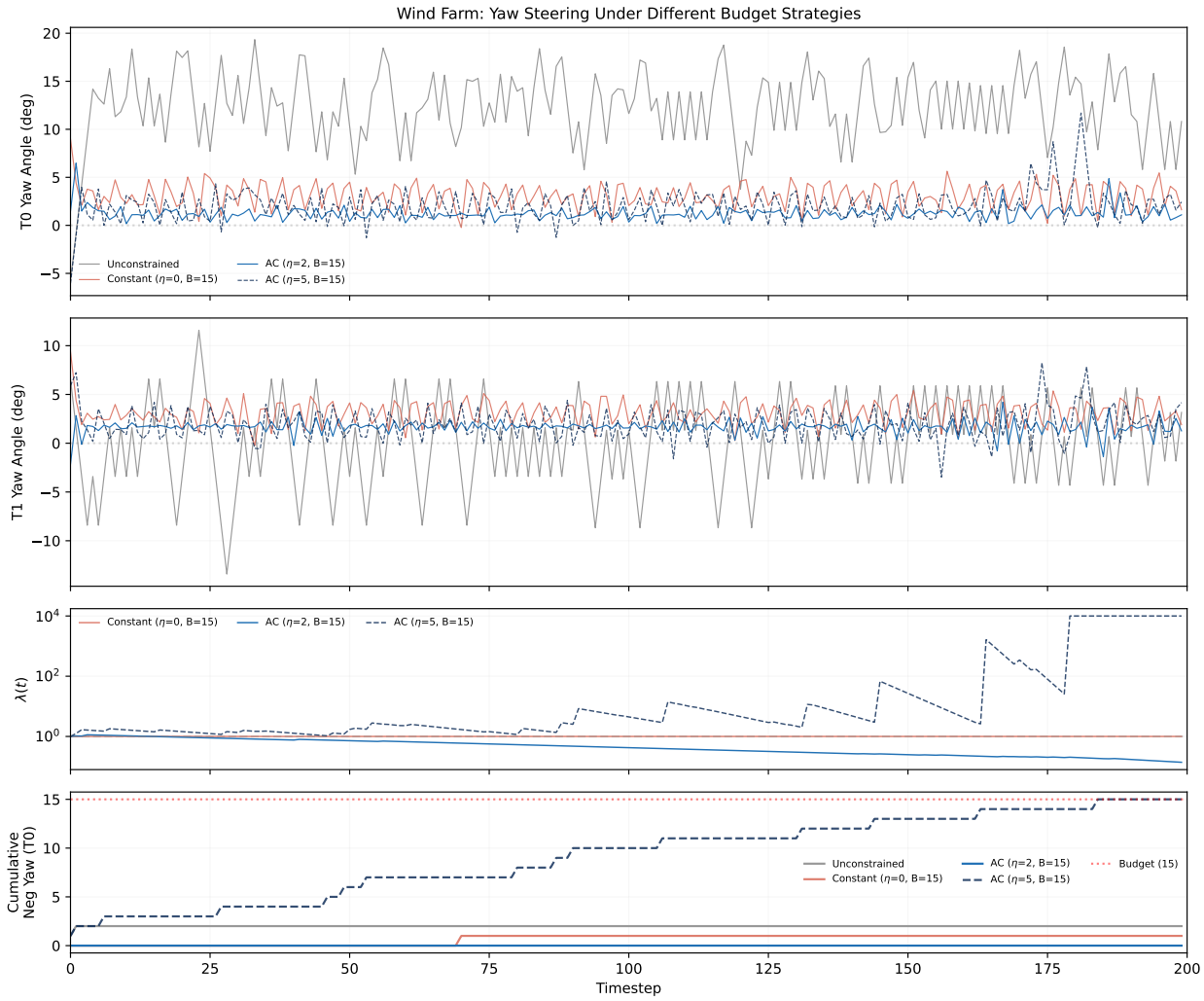

Figure 7: Wind farm yaw time series under different budget strategies (one episode, budget= 15). Top two panels: per-turbine yaw angles—unconstrained (gray) swings $\pm 20°$, while budget-constrained methods stay near 0–5°. Third panel: $\lambda(t)$ on log scale—constant $\eta = 0$ (red) stays flat; AC $\eta = 5$ (dashed) drops during surplus and spikes to 10^4 near budget depletion. Bottom: cumulative negative yaw—AC $\eta = 5$ uses the full budget by episode end; $\eta = 2$ conserves heavily.

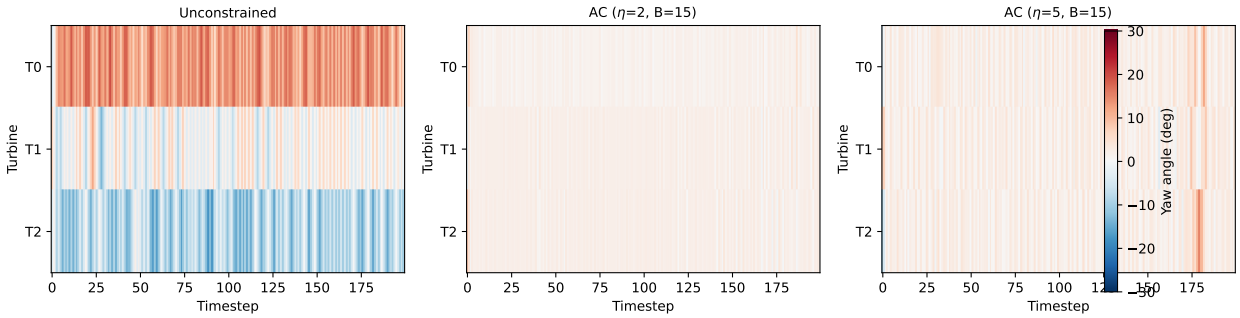

Figure 8: Yaw angle heatmaps (turbine × timestep) showing the spatial-temporal pattern. Unconstrained (left): persistent positive/negative yaw steering across all turbines. AC $\eta = 2$ (middle): nearly zero yaw—very conservative. AC $\eta = 5$ (right): mostly zero with a concentrated burst of negative yaw (blue) near the end, representing strategic budget spending.

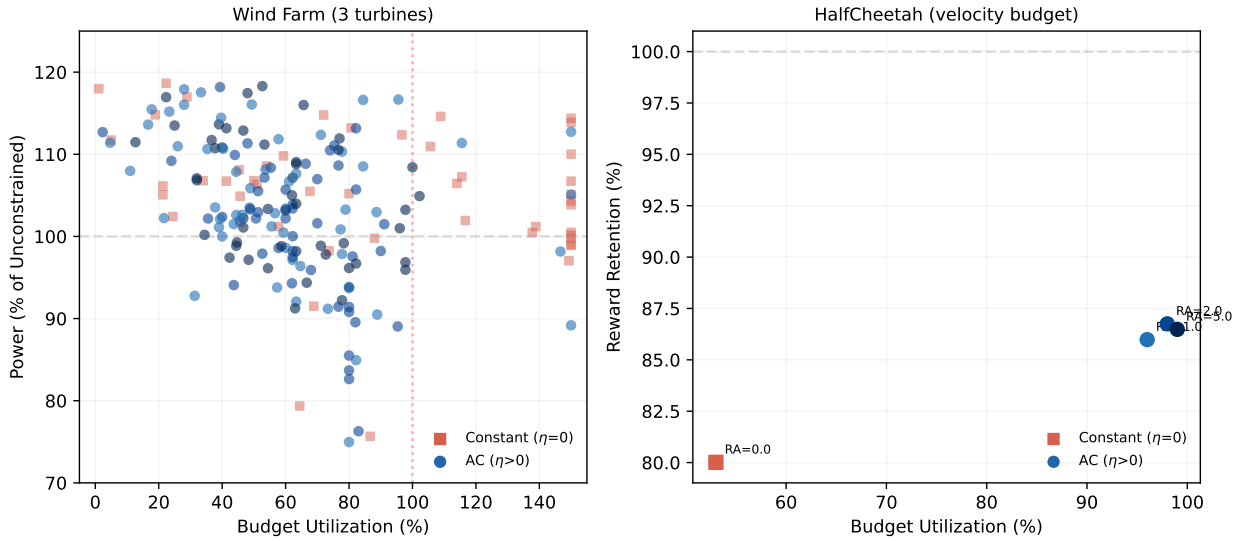

Figure 9: Budget utilization vs. reward retention. Constant penalty (squares, red) cannot simultaneously achieve high utilization and high reward. AC schedule (circles, blue shades by η) clusters in the efficient top-right region.

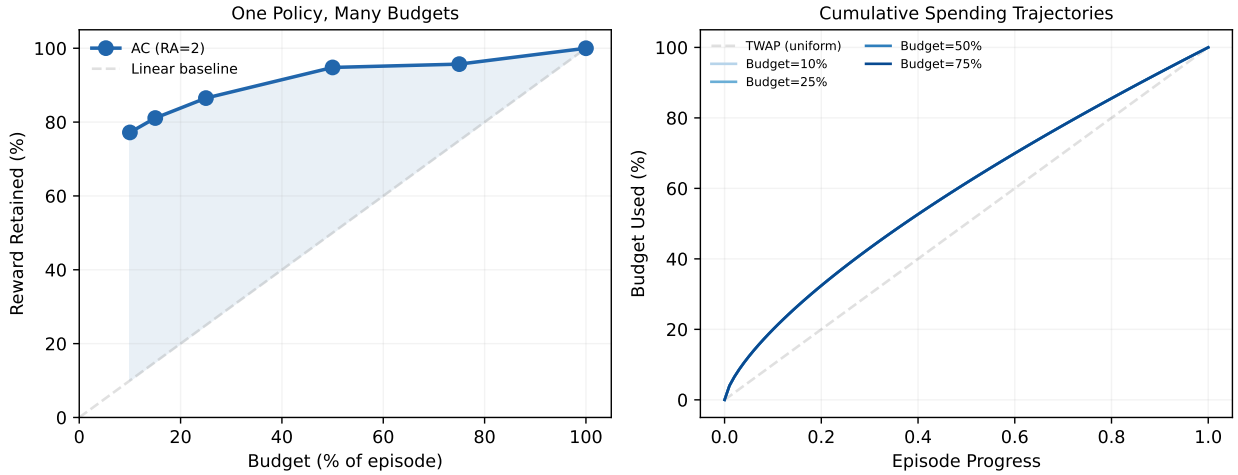

Figure 10: Budget flexibility. Left: same policy at different budget levels (concave curve = intelligent allocation). Right: cumulative spending trajectories depart from the TWAP diagonal, spending conservatively early and catching up during high-value phases.

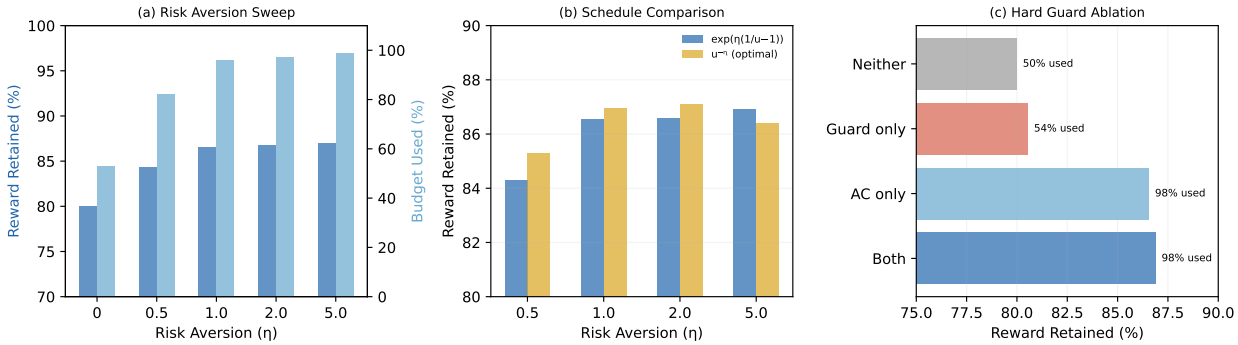

Figure 11: Ablations on HalfCheetah. (a) Risk aversion sweep: $\eta = 0$ wastes budget (53% used), $\eta \geq 1$ achieves >96% utilization and 7% more reward. (b) Schedule comparison: $u^{-\eta}$ (optimal) and $\exp(\eta(1/u - 1))$ (practical) perform comparably. (c) Hard guard ablation: the AC schedule alone matches the full system.